

Macronutrient Mass Prediction from Food Images

Senén Marqués

June 6, 2026

1 Abstract

2 Aim

The goal of this work is to systematically explore ways in which to predict the macronutrient mass (in grams) from food images. Macronutrient mass (grams of protein, fat and carbohydrate) were chosen as the three targets and they were used to calculate calories. Predictions were made from 1) a single image and 2) no reference object for size in the image. These rules served to limit the scope of the project, to allow for the use of the same dataset, and to make a general solution that is suitable for real world applications.

Achieving SOTA or close to SOTA quality presented a significant challenge. In order to accomplish this, the project adopted an experimental and research driven approach, using the literature to justify decisions and hypotheses, and testing those hypotheses through experimentation, thus adhering to the scientific method

3 The Data

3.1 Choosing a dataset

The dataset used was the **MM-Food-100K Dataset** [1], which contains ~100K labeled images of restaurant and home cooked food. It was chosen because the images are labelled with nutritional information needed for this work, for of its large size, and because of its diversity: featuring different dish sizes, cuisines, lighting, camera-quality ... etc.

Other researched datasets include ACEDATA [2], FooDD [3], Pittsburgh Fast Food Image Dataset [4] and Nutrition5K [5]. All of which, provided interesting insights into what makes a good dataset for the task in hand.

3.2 Loading the data

The first step was to download the *MM-Food-100K Dataset*, to do this, a python script was made to download the dataset csv file and images concurrently and idempotently, meaning that upon each execution, the

script would attempt to download any missing images, as it took many hours to download the whole 166 GB of images and it was unrealistic to expect the whole script to finish execution uninterrupted in one go. To ensure maximum speed, the data loading script featured concurrency for networking tasks and parallelism for processing and disk access.

After having a local copy of the dataset, a custom pytorch Dataset class was made to allow obtaining the image and targets when iterating over it. That Dataset class would be passed to a DataLoader and then fed to the model.

Later during training, a bottleneck was found, it was caused by the heavy image transform computations from the DataLoader loading batches into the model. The data loading script was modified to offload the image resizing computation to that data loading stage. After that improvement, batches were fed to the model during training at approximately twice the speed.

Finally, upon revisiting the code months later, it was evident that having almost two hundred gigabytes of images locally was not a good idea, so the data loading script was refactored and improved one final time so that images would be downloaded and resized in memory and then saved to disk. This cut the dataset size into a fraction of what it was before and improved the speed of the script somewhat as it made less I/O operations.

4 Multi-output Regression

The computer vision task at hand requires multi output regression. A Neural Network can be adapted for regression by modifying the head of the network and changing the criterion from Cross-Entropy loss to MSE (L2) or MAE (L1) Loss.

The literature on CNNs is very classification biased, as it is centered around benchmarking on ImageNet classification. After reading much literature on CNN architectures, the question of whether any ImageNet pretrained classification CNN would work well for our regression task emerged. [6] analysed how 16 model architectures perform when fine tuned, trained from scratch or used as feature extractors for 12 different datasets. They found that:

- Imagenet accuracy is a good indicator of transfer accuracy
- On some small, fine grained datasets, the benefit of fine-tuning vs training from scratch is minimal.
- The benefits of ImageNet pretraining fade for larger datasets.
- ImageNet pretraining accelerates convergence.

With this information, it was decided to use transfer learning when training the model, because even if the MMFood100K dataset is too large or too different from ImageNet, it will converge faster saving a of training time.

Sadly, [6] did not consider regression or predictions of continuous numerical values, and not much information was found on this topic besides some casual internet articles.

4.1 Model Architecture Evaluation

Before using transfer leaning to train a model to do multi-output regression, it was decided to make some empirical experiment to predict which model architecture would perform better.

To find which pretrained model architecture would transfer better with out dataset, or in other words, which would be the most suitable model for our data, a “*model shootout*” study was devised.

A study was made with the Optuna library that would pick different pretrained models using grid sampling, train the models with the same hyperparameters, for a fixed number of epochs using feature extraction (frozen backbone, training only the head).

This was a cheap and fast way to make an educated guess to pick the model since “*when [different models are] used as fixed feature extractors ..., ImageNet top-1 accuracy was correlated with accuracy on transfer learning*” [6] .

After runnign the “*model shootout*” optuna study for four times, it was determined that out of the following models:

- EfficientNet_B3
- EfficientNet_V2_S
- MobileNet_V3_L
- Swin_V2_S

The Swin came on top being the fastest learner, which makes sense since all the others are CNNs and Swin is a more modern Vision Transformer.

The variant of each model was chosen as the biggest variant that could fit on a 16GB VRAM RTX 3060 ti GPU.

4.2 Sequential fine tuning with Hyperparameter Optimization

After the Swin model won the shootout, a two step training stage was ran in an extensive hyperparameter optimization study to find the best settings for the model and minimize the loss.

The sequential fine tuning consisted of a *warm up* step before fine tuning, where head learns while the backbone is frozen.

The sequential fine tuning was ran in an extensive hyperparameter optimization study using the Optuna library, that optimized the following hyperparameters:

- Feature extraction phase learning rate
- Feature extraction weight decay
- Feature extraction epochs
- Fine tuning phase learning rate
- Fine tuning weight decay
- Fine tuning epochs
- Loss used (MAE, MSE or Huber)

And the following hidden flat layer that was inserted before the head

- number of hidden units before the head (not a hyperparameter)
- hidden layer dropout

5 LMM enriched with metadata

While researching the task at hand, [2] found that when feeding an image of food enriched with metadata like GPS coordinates, time of day and a list of ingredients to an LMM (Large Multimodal Model) at prompt time, while using prompting techniques like Chain of Thought, Scale Hint in the image, Few-shot and Expert Persona, the error in the predictions would shrink significantly. However, [2] reported MAE too big to make an accurate food logging system out of it.

A system was devised, that consisted of: - A pretrained food classifier - A coordinate fetching function - A timestamp function

6 Results

[I currently have no results to showcase] [I will update the document with more sections, references and result visualizations]

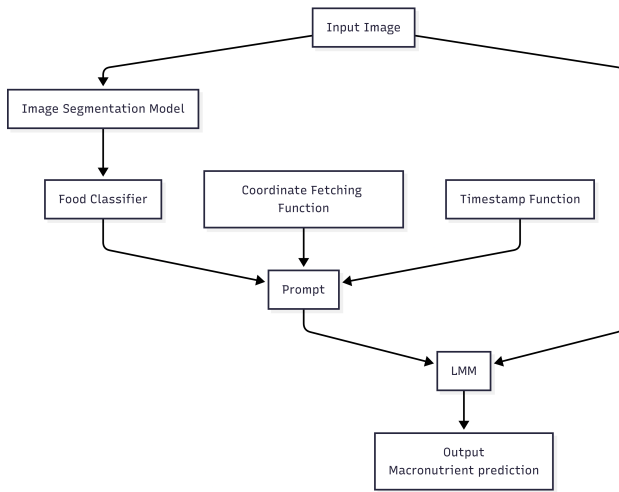


Figure 1: LMM enriched with metadata

References

- [1] Y. Dong, Y. Muraoka, S. Shi, and Y. Zhang, "MM-food-100K: A 100,000-sample multimodal food intelligence dataset with verifiable provenance." 2025. Available: <https://arxiv.org/abs/2508.10429>
- [2] B. Coburn, J. He, M. E. Rollo, S. S. Dhaliwal, D. A. Kerr, and F. Zhu, "Comprehensive evaluation of large multimodal models for nutrition analysis: A new benchmark enriched with contextual metadata." 2025. Available: <https://arxiv.org/abs/2507.07048>
- [3] P. Pouladzadeh, A. Yassine, and S. Shirmohammadi, "FooDD: Food detection dataset for calorie measurement using food images," *IEEE Transactions on Instrumentation and Measurement*, vol. 63, no. 8, pp. 1947–1956, 2014, doi: 10.1109/TIM.2014.2323337.
- [4] M. Chen, K. Dhingra, W. Wu, L. Yang, R. Sukthankar, and J. Yang, "PFID: Pittsburgh fast-food image dataset," Carnegie Mellon University, CMU-TR-06-09, 2009.
- [5] Q. Thames *et al.*, "Nutrition5k: Towards automatic nutritional understanding of generic food." 2021. Available: <https://arxiv.org/abs/2103.03375>
- [6] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" 2019. Available: <https://arxiv.org/abs/1805.08974>